7장 생성 모델

```
# 예제 7.1 오토 인코더(fashion_mnist 재구성)

#셋업
import tensorflow as tf
from tensorflow.keras import Model
from tensorflow.keras.layers import Dense, Flatten, Reshape
from tensorflow.keras.datasets import fashion_mnist
import numpy as np
import matplotlib.pyplot as plt
```

```
# 데이터셋 준비
(x_train, _), (x_test, _) = fashion_mnist.load_data()  # 레이블은 사용하지 않음
```

```
# 데이터 정규화
x_train = x_train / 255
x_test = x_test / 255
```

```
# 인코더 정의
class Encoder(Model):
    def __init__(self, latent_dim):
        super().__init__()
        self.flatten = Flatten()
        self.dense = Dense(units=latent_dim, activation='relu')

    def call(self, x):
        x = self.flatten(x)
        x = self.dense(x)
        return x
```

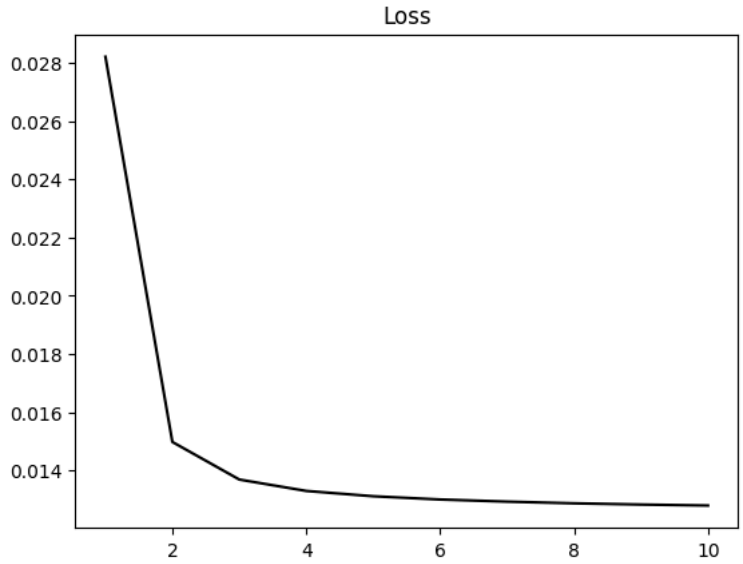
```
# 디코더 정의
class Decoder(Model):
    def __init__(self, latent_dim):
        super().__init__()
        self.dense = Dense(units=28 * 28, activation="sigmoid")
        self.reshape = Reshape((28, 28))

    def call(self, x):
        x = self.dense(x)
        x = self.reshape(x)
        return x
```

```
# 오토 인코더 정의
class AutoEncoder(Model):
    def __init__(self, latent_dim):
        super().__init__()
        self.encoder = Encoder(latent_dim)
        self.decoder = Decoder(latent_dim)

    def call(self, inputs):
        x = self.encoder(inputs)
        x = self.decoder(x)
        return x

# 모델 생성
model = AutoEncoder(latent_dim=32)
```

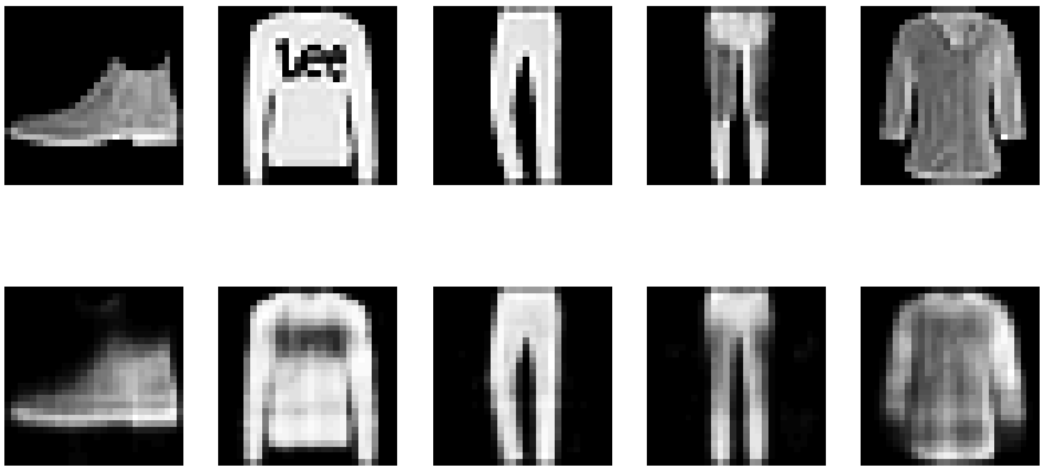
```
# 모델 컴파일
model.compile(optimizer="adam", loss="mse")
```

```
# 모델 학습
history = model.fit(x_train, x_train, epochs=10, verbose=0)
```

```
# 학습 결과 시각화
plt.plot(range(1, len(history.history["loss"]) + 1),
         history.history["loss"], color="black")
plt.title("Loss")

plt.show()
```




```
# 원본 이미지/재구성 이미지 시각화
plt.figure(figsize=(10, 5))

for i in range(5):

    # 원본 이미지
    plt.subplot(2, 5, i + 1)
    plt.imshow(x_test[i], cmap="gray")
    plt.axis("off")

    # 재구성 이미지
    plt.subplot(2, 5, i + 6)
    plt.imshow(model(x_test)[i], cmap="gray")
    plt.axis("off")

plt.show()
```



예제 7.2

https://www.kaggle.com/datasets/jessicali9530/celeba-dataset

```
# 예제 7.2 VAE 사람 얼굴 생성 소규모 CelebA(1k)

from google.colab import drive
drive.mount("/content/drive")
```

```
Mounted at /content/drive
```

```
# 셋업
import tensorflow as tf
from tensorflow.keras import Model, Input
from tensorflow.keras.layers import Conv2D, Conv2DTranspose, Layer
from tensorflow.keras.layers import Dense, Flatten, Reshape
from tensorflow.keras.datasets import mnist
import numpy as np
```

```
import matplotlib.pyplot as plt
import os
import PIL
from IPython import display
```

```
# 데이터셋 폴더 경로 지정
dir_path = "/content/drive/MyDrive/Datasets/celeba_1k"
```

```
# 원본 이미지 확인
file_path = os.path.join(dir_path, "000001.jpg")  # 파일 경로
image = PIL.Image.open(file_path)

print("size of image:", image.size)

plt.figure(figsize=(3, 3))
plt.imshow(image)
plt.axis("off")

plt.show()
```

```
size of image: (178, 218)
```

```
# 데이터셋 준비
train_ds = tf.keras.utils.image_dataset_from_directory(
    directory= dir_path,  # 폴더 경로 지정
    label_mode=None,  # 레이블 사용하지 않음
    image_size=(64, 64),  # 이미지 크기(64x64)
    batch_size=32)  # 배치 크기(32)
```

```
Found 1000 files.
```

```
# 학습 이미지 시각화
for imgs in train_ds.take(1):
    plt.figure(figsize=(5, 5))
    plt.subplots_adjust(wspace=0.01, hspace=0.01)

    for i in range(25):
        plt.subplot(5, 5, i + 1)
        plt.imshow(imgs[i] / 255)
        plt.axis("off")

    plt.show()
```

```
# 이미지를 넘파이 배열로 변환
x_train = []

for batch in train_ds:
    x_train.append(batch.numpy())

x_train = np.concatenate(x_train, axis=0)

print(x_train.shape)   # 학습 데이터 shape 확인
```

```
(1000, 64, 64, 3)
```

```
# 정규화
x_train = x_train / 255
```

```
# 인코더 정의
latent_dim = 2

inputs = Input(shape=(64, 64, 3))
x = Conv2D(32, kernel_size=3, strides=2, padding="same",
           activation="relu")(inputs)
x = Conv2D(64, kernel_size=3, strides=2, padding="same",
           activation="relu")(x)
x = Conv2D(128, kernel_size=3, strides=2, padding="same",
           activation="relu")(x)
x = Flatten()(x)
x = Dense(32, activation="relu")(x)

mu = Dense(latent_dim)(x)
log_var = Dense(latent_dim)(x)

encoder = Model(inputs, [mu, log_var])
```

```
# 잠재 공간 샘플링 계층 정의
class Sampling(Layer):
    def call(self, inputs):
        mu, log_var = inputs
        epsilon = tf.random.normal(tf.shape(log_var))
        return mu + tf.exp(0.5 * log_var) * epsilon
```

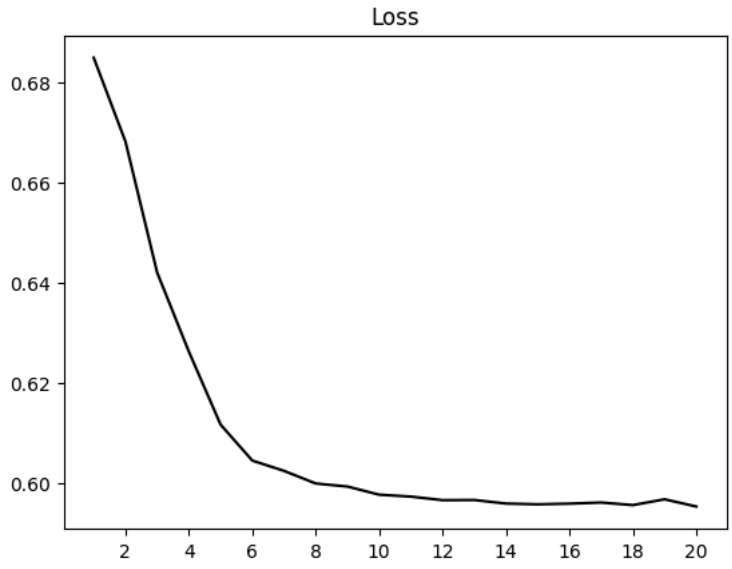
```
# 디코더 정의
decoder_inputs = Input(shape=(latent_dim,))
x = Dense(8 * 8 * 128, activation="relu")(decoder_inputs)
x = Reshape((8, 8, 128))(x)
x = Conv2DTranspose(128, kernel_size=3, strides=2, padding="same",
                    activation="relu")(x)
x = Conv2DTranspose(64, kernel_size=3, strides=2, padding="same",
                    activation="relu")(x)
x = Conv2DTranspose(32, kernel_size=3, strides=2, padding="same",
                    activation="relu")(x)
outputs = Conv2D(3, kernel_size=3, strides=1, padding="same",
                 activation="sigmoid")(x)

decoder = Model(decoder_inputs, outputs)
```

```
# 변형 오토 인코더 정의
class VAE(Model):
    def __init__(self, encoder, decoder):
        super(VAE, self).__init__()
        self.encoder = encoder
        self.decoder = decoder
        self.sampling = Sampling()

    def call(self, inputs):
        mu, log_var = self.encoder(inputs)
        z = self.sampling([mu, log_var])
        reconstruction = self.decoder(z)

        latent_loss = -0.5 * tf.reduce_sum(   # 잠재 손실
            1 + log_var - tf.exp(log_var) - tf.square(mu), axis=-1)
        latent_loss = tf.reduce_mean(latent_loss) / 784   # 픽셀 당 잠재 손실

        self.add_loss(latent_loss)   # 손실 = 재구성 손실 + 잠재 손실
        return reconstruction

model = VAE(encoder, decoder)
```

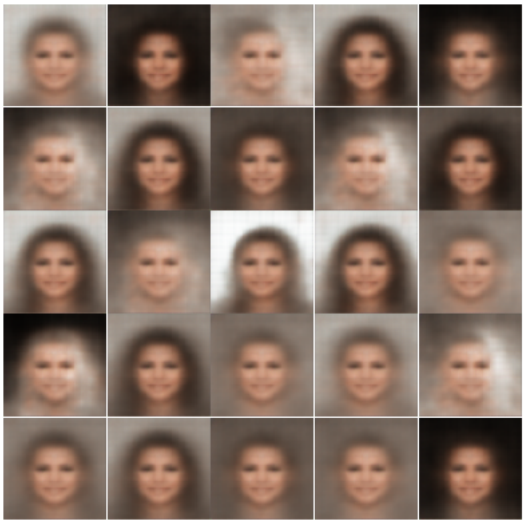
```
# 모델 컴파일
model.compile(optimizer="adam", loss="binary_crossentropy")
```

```
# 모델 학습
history = model.fit(x_train, x_train, epochs=20, verbose=0)
```

```
# 학습 결과 시각화
plt.plot(range(1, len(history.history["loss"]) + 1),
         history.history["loss"], color="black")
plt.title("Loss")
plt.xticks(range(2, len(history.history["loss"]) + 1, 2))

plt.show()
```




```
# 이미지 생성
n = 25  # 이미지 수

generated_images = []
for i in range(n):
    latent_point = np.random.normal(size=(1, 2))  # 잠재 공간의 점
    decoded_image = decoder.predict(latent_point, verbose=0)
    decoded_image = decoded_image[0].reshape(64, 64,3)
    generated_images.append(decoded_image)

# 생성 이미지 시각화
plt.figure(figsize=(5, 5))
plt.subplots_adjust(wspace=0.01, hspace=0.01)

for i in range(25):
    plt.subplot(5, 5, i+1)
    plt.imshow(generated_images[i])
    plt.axis("off")

plt.show()
```



```
# 예제 7.3 DCGAN 필기체 숫자 생성 MNIST

# 셋업
import tensorflow as tf
from tensorflow.keras import Model, Input
from tensorflow.keras.layers import Conv2D, Conv2DTranspose
from tensorflow.keras.layers import BatchNormalization, ReLU, LeakyReLU
from tensorflow.keras.layers import Dense, Reshape, Flatten, Dropout
from tensorflow.keras.datasets import mnist
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image
from IPython import display
```

```
# 데이터셋 준비
(x_train, _), (_, _) = mnist.load_data()  # 레이블/테스트 데이터는 사용하지않음
```

```
# 데이터 정규화(-1 ~ 1)
x_train = (x_train.astype(np.float32) - 127.5 ) / 127.5

# 2차원 이미지를 3차원으로 변환
x_train = x_train.reshape(x_train.shape[0], 28, 28, 1)

print(x_train.shape)
```

```
(60000, 28, 28, 1)
```

```
# 소규모 MNIST 데이터셋 생성
x_train = tf.data.Dataset.from_tensor_slices(x_train).batch(32)

x_train = x_train.take(200)  # 6,400개(200x32) 샘플
```

```
# 생성자 정의
generator_inputs = Input(shape=(100, ))  # 랜덤 벡터
x = Dense(7 * 7 * 128)(generator_inputs)
x = BatchNormalization()(x)
x = ReLU()(x)
x = Reshape((7, 7, 128))(x)

x = Conv2DTranspose(64, kernel_size=4, strides=2, padding="same")(x)
x = BatchNormalization()(x)
x = ReLU()(x)

x = Conv2DTranspose(32, kernel_size=4, strides=2, padding="same")(x)
x = BatchNormalization()(x)
x = ReLU()(x)

generator_outputs = Conv2DTranspose(1, kernel_size=4, strides=1, padding="same",
                    activation="tanh")(x)

generator = Model(generator_inputs, generator_outputs, name="generator")
```

```
# 판별자 정의
discriminator_inputs = Input(shape=(28, 28, 1))
x = Conv2D(32, kernel_size=4, strides=2, padding="same")(discriminator_inputs)
x = LeakyReLU(negative_slope=0.2)(x)

x = Conv2D(64, kernel_size=4, strides=2, padding="same")(x)
x = BatchNormalization()(x)
x = LeakyReLU(negative_slope=0.2)(x)

x = Conv2D(128, kernel_size=4, strides=2, padding="same")(x)
x = BatchNormalization()(x)
x = LeakyReLU(negative_slope=0.2)(x)

x = Flatten()(x)
x = Dropout(0.2)(x)
discriminator_outputs = Dense(1, activation="sigmoid")(x)

discriminator = Model(discriminator_inputs, discriminator_outputs,
                      name="discriminator")
```

```
# 하이퍼파라미터 설정
batch_size = 32  # 배치 크기
latent_dim = 100  # 랜덤 벡터 차원
num_generate = 100  # 생성할 이미지 수
seed = tf.random.normal([num_generate, latent_dim])  # 생성자 학습 시작 입력
```

```
# 손실 함수 설정
bce_loss = tf.keras.losses.BinaryCrossentropy()

# 옵티마이저 설정
g_optimizer = tf.keras.optimizers.Adam(learning_rate=0.0002, beta_1=0.5)
d_optimizer = tf.keras.optimizers.Adam(learning_rate=0.0002, beta_1=0.5)
```

```
# 학습 루프 함수 정의
@tf.function  # 함수를 그래프로 변환하여 속도 향상
def train_step(images):
    noise = tf.random.normal([batch_size, latent_dim])  # 랜덤 벡터

    # 판별자 학습
    with tf.GradientTape() as tape:  # 자동 미분 수행
        real_output = discriminator(images, training=True)
        fake_images = generator(noise, training=True)
        fake_output = discriminator(fake_images, training=True)

        real_loss = bce_loss(tf.ones_like(real_output), real_output)
        fake_loss = bce_loss(tf.zeros_like(fake_output), fake_output)
        d_loss = real_loss + fake_loss

    d_gradient = tape.gradient(d_loss, discriminator.trainable_variables)
    d_optimizer.apply_gradients(
        zip(d_gradient, discriminator.trainable_variables))

    # 생성자 학습
    with tf.GradientTape() as tape:  # 자동 미분 수행
        fake_images = generator(noise, training=True)
        fake_output = discriminator(fake_images, training=True)
        g_loss = bce_loss(tf.ones_like(fake_output), fake_output)

    g_gradient = tape.gradient(g_loss, generator.trainable_variables)
    g_optimizer.apply_gradients(
        zip(g_gradient, generator.trainable_variables))
```

```
# 이미지 생성/저장 함수 정의
def generate_save_images(model, epoch, inputs):
    prediction = model(inputs, training=False)
    min = tf.reduce_min(prediction)
    max = tf.reduce_max(prediction)
    prediction = (prediction - min) / (max - min)  # 픽셀 값(0 ~ 1)

    plt.figure(figsize=(5, 5))
    plt.subplots_adjust(wspace=0.01, hspace=0.01)

    for i in range(prediction.shape[0]):
        plt.subplot(10, 10, i + 1)
        plt.imshow(prediction[i], cmap="gray")
        plt.axis("off")
    plt.savefig(f"image at epoch {epoch}.png")
    plt.show()
```

```
# 학습 함수 정의
def train(dataset, epochs):
    for epoch in range(epochs):
        for image_batch in dataset:
            train_step(image_batch)

        # 에포크마다 이미지 생성
        display.clear_output(wait=True)

        print(f"Generated Images at Epoch {epoch + 1}")

        generate_save_images(generator, epoch + 1, seed)
```

```
# 학습(이미지 생성)
train(x_train, epochs=50)
```

```
Generated Images at Epoch 50
```

예제 7.4

https://commons.wikimedia.org/wiki/File:VanGogh-starry_night_ballance1.jpg

https://commons.wikimedia.org/wiki/File:Kandinsky_-_Composition_8,_1923.jpg

```
# 예제 7.4 스타일 전이(TensorFlowHUb)

from google.colab import drive
drive.mount("/content/drive")
```

```
Mounted at /content/drive
```

```
# 셋업
import tensorflow as tf
import tensorflow_hub as hub
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image
```

```
# 콘텐츠 이미지(사진)/스타일 이미지(칸딘스키 구성8) 준비
content_path = "/content/drive/MyDrive/Datasets/풍차.JPG"
content_image = Image.open(content_path)

style_path = "/content/drive/MyDrive/Datasets/칸딘스키_구성8.jpg"
style_image = Image.open(style_path)

print(f"size of content image: {content_image.size}")
print(f"size of style image: {style_image.size}")
```

```
size of content image: (1920, 1080)
size of style image: (980, 682)
```

```
# 콘텐츠 이미지/스타일 이미지 시각화
plt.figure(figsize=(10, 10))

plt.subplot(1, 2, 1)
plt.imshow(content_image)
plt.title("Content Image(Photo)")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(style_image)
plt.title("Style Image(Kandinsky Composition 8)")
plt.axis("off")

plt.show()
```


```
# 스타일 이미지 크기 조정
width, height = content_image.size
style_image = tf.image.resize(style_image, (height, width))

# 이미지 배열로 변환하고 정규화
content_image = np.array(content_image, dtype=np.float32) / 255.
style_image = np.array(style_image, dtype=np.float32) / 255.

# 배치 차원 추가
content_image = np.expand_dims(content_image, axis=0)
style_image = np.expand_dims(style_image, axis=0)

print(f"shape of content image: {content_image.shape}")
print(f"shape of style image: {style_image.shape}")
```

```
shape of content image: (1, 1080, 1920, 3)
shape of style image: (1, 1080, 1920, 3)
```

```
# 스타일 전이 모델 다운로드
url = "https://tfhub.dev/google/magenta/arbitrary-image-stylization-v1-256/2"

model = hub.load(url)
```

```
# 스타일 전이 이미지 생성
outputs = model(tf.constant(content_image), tf.constant(style_image))
stylized_image = outputs[0]  # 출력 리스트에서 첫 번째 텐서 선택
stylized_image = tf.squeeze(stylized_image, axis=0)  # 배치 차원 축소
```

```
# 스타일 전이 이미지 시각화
plt.figure(figsize=(7, 7))

plt.imshow(stylized_image)
plt.title("Stylized Image")
plt.axis("off")

plt.show()
```


```
# 예제 7.5 이미지 생성(KerasCV 스테이블 디퓨전)

# KerasCV 설치
!pip install keras-cv --quiet
```

```
# 스테이블 디퓨전 모델 불러오기
import keras_cv

model = keras_cv.models.StableDiffusion(img_width=512, img_height=512)
```


```
# 스테이블 디퓨전 모델 불러오기
import keras_cv

model = keras_cv.models.StableDiffusion(img_width=512, img_height=512)
```


```
By using this model checkpoint, you acknowledge that its usage is subject to the terms of the CreativeML Open RAIL-M license at https://
```

```
# 이미지 생성
prompt = "A small cabin on top of a snowy mountain"

images = model.text_to_image(prompt, batch_size=2)
```

```
50/50 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 59s 1s/step
```

```
# 생성 이미지 시각화
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 10))

for i, img in enumerate(images):
    plt.subplot(1, len(images), i + 1)
    plt.imshow(img)
    plt.axis("off")

plt.show()
```

```
# 예제 7.5 이미지 생성(허깅페이스 Stable Diffusion)

# 허깅페이스 Diffuser 설치
!pip install diffusers
```

```
# 이미지 생성 파이프라인
from diffusers import StableDiffusionPipeline

token = "hg_coh_123"   # 토큰 입력

generator = StableDiffusionPipeline.from_pretrained(
    "CompVis/stable-diffusion-v1-4").to(device="cuda")   # GPU 필수
```

```
# 이미지 생성
prompt = "Pikachu flying on a cloud"

output = generator(prompt, num_images_per_prompt=2)
images = output.images   # PIL 이미지 추출
```

```
# 생성 이미지 시각화
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 10))

for i, img in enumerate(images):
    plt.subplot(1, len(images), i + 1)
```

```
        plt.imshow(img)
        plt.axis("off")

plt.show()
```

```
100%                                                  50/50 [00:44<00:00,  1.14it/s]
```